

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК И КИБЕРБЕЗОПАСНОСТИ
ВЫСШАЯ ШКОЛА ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

**Отчет о прохождении научно-исследовательской (производственной)
практики**

Киричек Марии Романовны

2 курс обучения , гр. 5130203/40001

02.03.03 Математическое обеспечение и администрирование информационных систем

Место прохождения практики: СПбПУ, ИКНК, ВШ ТИИ

Сроки практики: с 15.06.2026 по 11.07.2026.

Руководитель практики от ФГАОУ ВО «СПбПУ»: Пак Вадим Геннадьевич,
к.ф.-м.н., доцент ВШТИИ.

Оценка:

Руководитель практики
от ФГАОУ ВО «СПбПУ»

В.Г. Пак

Обучающийся

М.Р. Киричек

Дата: 11.07.2026

СОДЕРЖАНИЕ

Введение	3
Глава 1. Анализ методов и современных систем машинного перевода.....	5
1.1 История развития нейронного машинного перевода.....	5
1.1.1 Статистические методы	5
1.1.2 Рекуррентные нейронные сети	6
1.2 Сравнение рекуррентных нейронных сетей и архитектуры Transformer	6
1.3 Современные системы	7
1.4 Выводы	7
Глава 2. Теоретические основы архитектуры Transformer	7
2.5 Подготовка входных данных	7
2.5.1 Токенизация	8
2.5.2 Слой Embedding	8
2.5.3 Позиционное кодирование.....	9
2.6 Архитектура Transformer	10
2.7 Остаточные связи и нормализация слоёв	12
2.8 Полносвязная нейронная сеть	14
2.9 Слои Linear и Softmax	15
2.10 Механизмы внимания	16
2.10.1 Self-attention	18
2.10.2 Multi-head attention.....	18
2.10.3 Encoder-decoder attention.....	19
2.11 Выводы	19
Глава 3. Проектирование, реализация и обучение модели Transformer	20
3.12 Подготовка корпуса OPUS-100 и предобработка данных	20
3.13 Реализация архитектуры Transformer на PyTorch	22
3.14 Обучение модели	23
3.15 Процесс генерации перевода	24
3.16 Выводы	25
Глава 4. Анализ качества перевода обученной модели	25
4.17 Оценка качества перевода	25
4.18 Выводы	27
Заключение	28
Список использованных источников.....	29
Приложение 1 Листинг программного кода.....	31

ВВЕДЕНИЕ

Задача машинного перевода является одной из важнейших задач в области естественной обработки языка (Natural Language Processing, NLP). В современном мире постоянно возникают задачи перевода текста с одного языка на другой в различных сферах деятельности от бытового общения до, например, инженерной и проектной документации.

В 2017 году исследователи из Google предложили в своей статье «Attention is all you need» архитектуру модели искусственной нейронной сети (ИНС) "Transformer". Отличительной особенностью их работы стала разработка первой модели, в основе которой они ушли от традиционных рекуррентных слоёв, используемых в encoder-decoder архитектурах, к механизмам внимания [4]. Данная архитектура лежит в основе современных систем искусственного интеллекта (СИИ), например, в системах машинного перевода текста и диалоговых агентах (например, Claude Sonnet). Часто в задачах машинного перевода с одного языка на другой СИИ могут быть представлены диалоговыми ассистентами или ИИ-агентами [7].

Актуальность исследования заключается в необходимости постоянного развития и совершенствования методов машинного перевода. В частности, к такому методу и относится архитектура Transformer, которая будет реализована в данной работе для машинного русско-английского перевода текста. Понимание механизмов работы этой архитектуры остаётся важным для формирования фундаментальных навыков проектирования нейросетевых архитектур.

Объект исследования — сфера машинного перевода текстов с русского языка на английский.

Предмет исследования — применение моделей с архитектурой Transformer, её компоненты и методы оценки качества для машинного перевода.

Цель исследования — реализовать и экспериментально исследовать архитектуру Transformer для задачи русско-английского машинного перевода.

В соответствии с целью исследования, логически определяются следующие **задачи работы**:

- А. Изучить архитектуру Transformer.
- В. Выполнить подготовку русско-английского датасета OPUS-100: провести токенизацию, нормализацию текста и фильтрацию выбросов, разбиение на обучающую, валидационную и тестовую выборки.

- C. Реализовать архитектуру Transformer на фреймворке PyTorch.
- D. Провести обучение модели, экспериментальное исследование влияния гиперпараметров на качество перевода, измеряемое метриками BLEU, METEOR и TER.
- E. Выполнить тестирование обученной модели, провести анализ ошибок перевода.
- F. Сформулировать выводы о применимости полученных знаний и навыков для разработки диалогового ассистента для перевода текстов с русского языка на английский.

Исходный код реализации, включая полный Jupyter-ноутбук, доступен в открытом репозитории [1].

ГЛАВА 1. АНАЛИЗ МЕТОДОВ И СОВРЕМЕННЫХ СИСТЕМ МАШИННОГО ПЕРЕВОДА

Глава посвящена истории развития машинного перевода (параграф 1.1), сравнению архитектур RNN и Transformer (параграф 1.2), а также обзору современных систем (параграф 1.3).

1.1 История развития нейронного машинного перевода

1.1.1 Статистические методы

Первые системы машинного перевода, использовали лингвистические правила. Однако со временем возникла необходимость массово переводить международные новости и техническую документацию. Системы, основанные на лингвистических правилах, не справлялись с такими масштабами, составление словарей вручную требовало слишком много времени. Это привело к переходу на статистический машинный перевод, который работал на основе автоматического анализа больших параллельных корпусов данных [10; 12; 17].

Математической основой стандартного статистического машинного перевода служит фундаментальное уравнение машинного перевода, основанное на теореме Байеса (1.1).

$$\hat{e} = \arg \max_e (P(f|e) \cdot P(e)), \quad (1.1)$$

где:

- $\hat{e}$ — наилучший перевод;
- f — исходное предложение;
- e — предложение-кандидат на целевом языке;
- $P(f|e)$ — правдоподобие перевода, оценивающее точность передачи смысла исходного предложения;
- $P(e)$ — языковая естественность, оценивающая грамматическую корректность и плавность предложения на целевом языке [11];
- $\arg \max_e$ — оператор, выбирающий такое предложение e , при котором произведение вероятностей максимально [11].

Главным недостатком статистического машинного перевода было то, что модели не умели правильно согласовывать падежи, роды и окончания, из-за чего в готовом тексте часто возникали грамматические ошибки.

1.1.2 Рекуррентные нейронные сети

Развитие машинный перевод получил благодаря внедрению архитектуры Encoder-Decoder на базе рекуррентных нейронных сетей [13; 14; 17].

В рамках данной концепции кодировщик на основе блоков LSTM (Long Short-Term Memory — долгая краткосрочная память) или GRU (Gated Recurrent Unit — управляемый рекуррентный блок) сжимает входную последовательность в вектор контекста фиксированной размерности, а декодировщик пошагово генерирует целевой текст [8; 15; 16]. Ключевым ограничением таких систем являлось ухудшение качества перевода при увеличении длины предложений [14; 18].

1.2 Сравнение рекуррентных нейронных сетей и архитектуры Transformer

Рекуррентные сети по принципу своей работы обрабатывают входную последовательность "слово за словом". В архитектуре Transformer это работает иначе, благодаря механизму внимания последовательность обрабатывается целиком.

По данным из таблицы 1.1, взятой из статьи "The attention is all you need" [4], информация в рекуррентном слое проходит через n шагов, равных длине последовательности, когда в Self-Attention шаг всего один. За эти n шагов в рекуррентном слое информация затухает, поэтому модель со временем теряет контекст. По этой причине сейчас стандартом в нейронном машинном переводе является архитектура Transformer [4].

Таблица 1.1

Максимальная длина пути, сложность на слой и минимальное число последовательных операций для разных типов слоёв [4]

Тип слоя	Сложность на слой	Последовательные операции	Максимальная длина пути
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$

1.3 Современные системы

Системы, которые сейчас используют для перевода текста, можно разделить на несколько групп [13]. Первая группа это мультязычные модели — сети, такие как mBART, которые обучаются на сотнях языков одновременно и используют перенос знаний — transfer learning. Их преимущество — способность переводить между редкими языковыми парами. Однако они уступают системам второй группы в точности формулировок.

Вторая группа это специализированные модели, обученные целенаправленно на русско-английских корпусах, например MarianMT.

И, последняя, третья группа — коммерческие системы — платформы от крупных технологических компаний, ориентированные на массового пользователя. К примеру, Яндекс.Переводчик и Google Translate.

1.4 Выводы

В главе рассмотрена эволюция машинного перевода от лингвистических правил до нейронных архитектур. Статистические методы позволили автоматизировать перевод, но не учитывали грамматику. Рекуррентные сети решали эту проблему, но теряли контекст в длинных предложениях. Архитектура Transformer обеспечивает лучшее качество перевода и это делает её технологическим стандартом.

ГЛАВА 2. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АРХИТЕКТУРЫ TRANSFORMER

В главе рассматриваются теоретические основы архитектуры Transformer: подготовка входных данных (2.5), структура модели (2.6), остаточные связи и нормализация (2.7), полносвязные сети (2.8), выходные слои (2.9) и механизмы внимания (2.10).

2.5 Подготовка входных данных

Перед тем как текст может быть обработан архитектурой Transformer, он должен быть обработан. Поскольку нейронные сети способны работать только с

числовыми данными, исходный текст преобразуется в последовательность векторов. Данный процесс включает три основных этапа: токенизацию, преобразование токенов в эмбединги и добавление информации о положении каждого токена в последовательности с помощью позиционного кодирования.

2.5.1 Токенизация

Токенизация представляет собой процесс разбиения текста на отдельные элементы — токены, которые являются входными данными для модели. В простейшем случае токенами могут быть слова, однако это приводит к созданию большого словаря и невозможности перевода неизвестных слов.

Для решения данной проблемы применяются алгоритмы субсловной токенизации [9]. Вместо хранения каждого слова целиком модель разбивает слова на более мелкие фрагменты (субслова, подслова), из которых впоследствии могут быть составлены как известные, так и ранее не встречавшиеся слова. Например, редкое для модели слово «подводный» алгоритм разобьет на части: ["под "##вод "##ный"]. В итоге модель понимает смысл слова по его частям, даже если никогда не видела его целиком.

Одним из наиболее распространённых методов является алгоритм Byte Pair Encoding. Первоначально каждое слово представляется последовательностью отдельных символов. Затем наиболее часто встречающиеся соседние пары символов постепенно объединяются в новые токены. После большого числа итераций формируется словарь наиболее часто встречающихся подслов.

Другим популярным методом является SentencePiece. В отличие от BPE, данный алгоритм работает непосредственно с необработанным текстом и не требует предварительного деления предложений на слова. Благодаря этому SentencePiece получил широкое распространение в современных языковых моделях.

2.5.2 Слой Embedding

После выполнения токенизации каждый полученный токен заменяется числовым идентификатором, который затем преобразуется в вектор с помощью слоя Embedding. Данный слой представляет собой обучаемую матрицу параметров, в которой каждому токenu словаря соответствует собственный вектор признаков.

Пусть размер словаря равен $|V|$, а размерность пространства признаков — d_{model} . Тогда матрица эмбедингов имеет размерность $W_E \in \mathbb{R}^{|V| \times d_{model}}$. Для каждого токена с индексом x_i выбирается соответствующая строка матрицы $e_i = W_E[x_i]$. В оригинальной архитектуре Transformer $d_{model} = 512$.

Следует отметить, что значения эмбедингов не задаются заранее. Они обучаются совместно со всеми остальными параметрами модели методом обратного распространения ошибки. Благодаря этому слова, имеющие схожее значение или употребляющиеся в похожем контексте, получают близкие векторные представления.



Рис.2.1. Преобразование токенов в эмбединги.

2.5.3 Позиционное кодирование

Архитектура Transformer одновременно обрабатывает всю последовательность токенов и не обладает информацией об их порядке. Поэтому к эмбедингам дополнительно добавляется позиционное кодирование, позволяющее модели учитывать расположение слов в предложении. В оригинальной работе используется синусоидальное позиционное кодирование, вычисляемое по формулам 2.2 и 2.3.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right); \quad (2.2)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right). \quad (2.3)$$

В них pos — позиция токена в последовательности, а i — индекс компоненты вектора. Чётные индексы ($2i$) получают значение синуса. Нечётные индексы ($2i+1$)

получают значение косинуса. Чем больше i , тем меньше частота и медленнее меняется функция. Это позволяет модели различать как близкие, так и далёкие позиции. После вычисления позиционный вектор складывается с соответствующим эмбедингом по формуле (2.4).

$$X = E + PE. \quad (2.4)$$

Именно полученные после этого представления поступают на вход первого слоя кодировщика Transformer. Это представлено на рисунке 2.2.



Рис.2.2. Добавление позиционного кодирования к эмбедингам.

Благодаря этому модель получает информацию не только о содержании токенов, но и об их относительном положении в последовательности.

2.6 Архитектура Transformer

Для начала рассмотрим архитектуру Transformer на абстрактном уровне, не углубляясь во внутреннее устройство её компонентов. Для этого посмотрим на модель как на чёрный ящик. В приложении для перевода с одного языка на другой на ввод будет подаваться предложение на первом языке, а на выводе будет оно же, переведённое на второй язык. Это показано на рисунке 2.3.

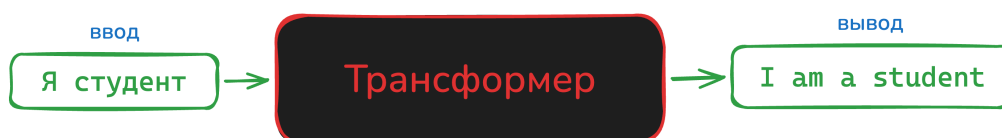


Рис.2.3. Архитектура Transformer [3].

При более детальном рассмотрении архитектуры можно выделить компонент кодирования, компонент декодирования и связи между ними.

Компонент кодирования представляет собой стек кодировщиков (в статье "The attention is all you need"[4] их 6), рис. 2.4. Компонент декодирования представляет собой стек декодеров. Между кодерами и декодерами есть связи.

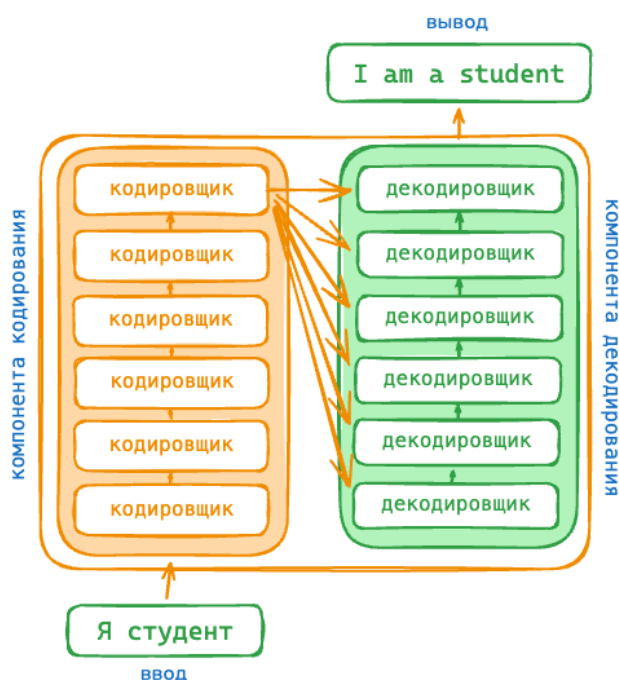


Рис.2.4. Более подробная архитектура Transformer [3].

Рассмотрим подробнее структуру кодировщика. Все кодировщики идентичны по структуре. Каждый кодировщик состоит из нескольких функциональных блоков, каждый из которых выполняет определённую задачу при обработке входной последовательности. Входные данные кодировщика сначала проходят через self-attention (самовнимание) — слой, который помогает кодировщику просматривать другие слова в предложении, которое подаётся на ввод, при кодировании определенного слова. Выходные данные слоя self-attention передаются в нейронную сеть feed forward (это простейший многослойный перцептрон, в нём информация движется от входа к выходу, прямо без циклов и обратных связей). Одна и та же сеть feed forward независимо применяется к каждой позиции, рис. 2.5. Строение данных двух слоёв будет подробно рассматриваться в следующих параграфах [4].

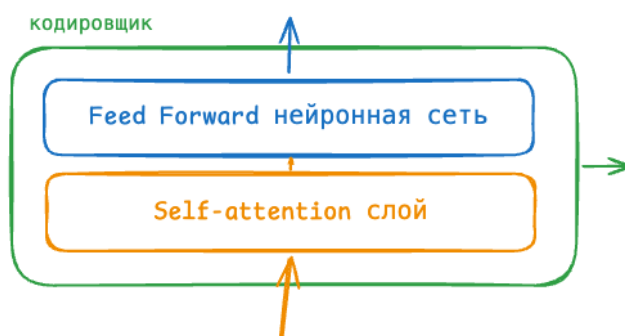


Рис.2.5. Кодировщик [3].

Теперь рассмотрим декодер. В декодере есть оба этих слоя, но между ними находится слой внимания, который помогает декодеру сосредоточиться на соответствующих частях входного предложения, рис. 2.6.

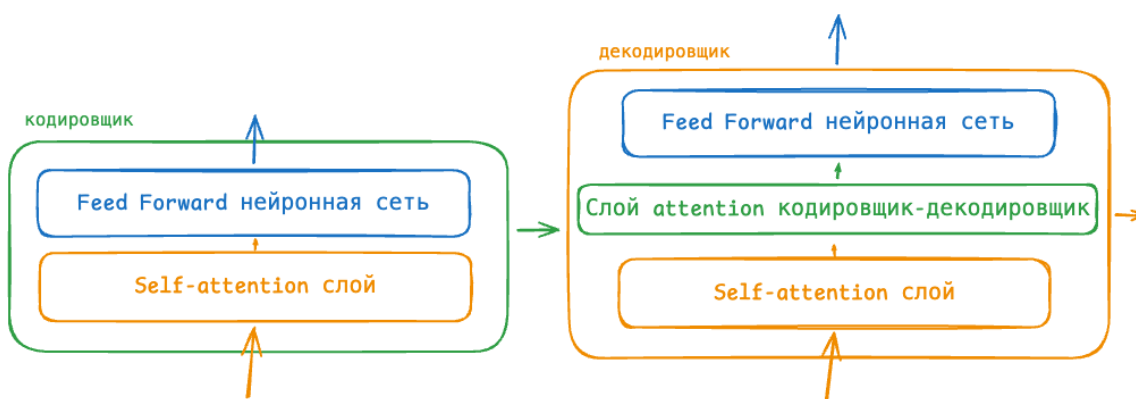


Рис.2.6. Кодировщик-декодировщик [3].

Для корректной работы каждого блока архитектуры используются дополнительные механизмы, обеспечивающие устойчивость и эффективность обучения модели. Одним из таких механизмов является операция *Add & Norm*.

2.7 Остаточные связи и нормализация слоёв

В оригинальной архитектуре Transformer после каждого подслоя применяется операция *Add & Norm*, которая описывается формулой:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)), \quad (2.5)$$

где x — вход в подслой, а $\text{Sublayer}(x)$ — выход подслоя (механизма внимания или полносвязной сети).

Остаточная связь $x + \text{Sublayer}(x)$ позволяет градиентам распространяться через сеть напрямую, решая проблему затухающих градиентов. Слой нормализа-

ции LayerNorm стабилизирует обучение, приводя активации к нулевому среднему и единичной дисперсии [4]. Архитектура кодировщика с данными слоями представлена на рисунке 2.7.

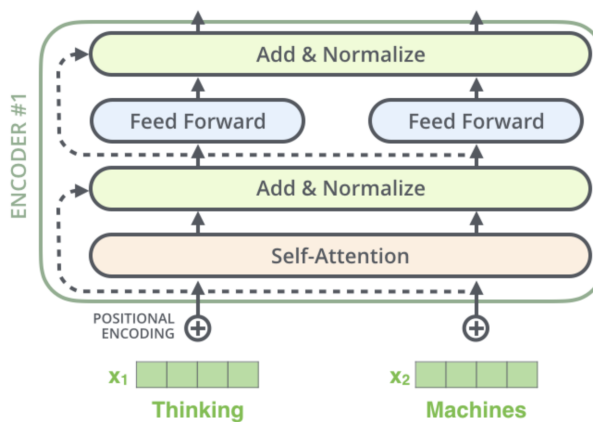


Рис.2.7. Кодировщик со слоями Add & Norm [3].

Архитектура кодировщика с данными слоями, но уже более подробно, представлена на рисунке 2.8.

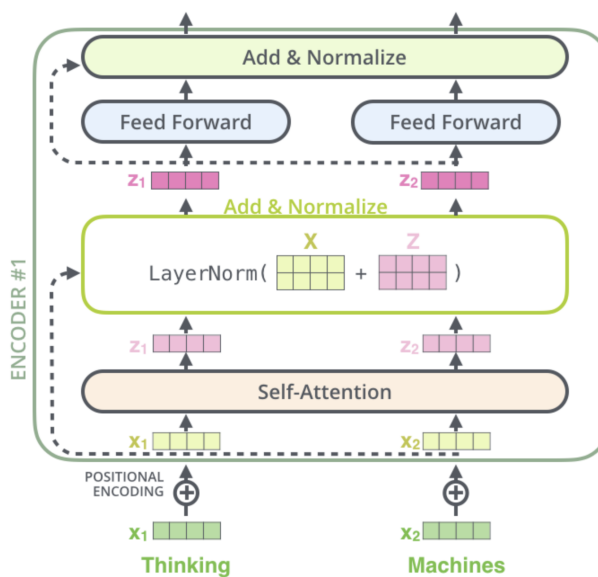


Рис.2.8. Слои Add & Norm подробно [3].

Полная архитектура модели с кодировщиком и декодировщиком представлена на рисунке 2.9.

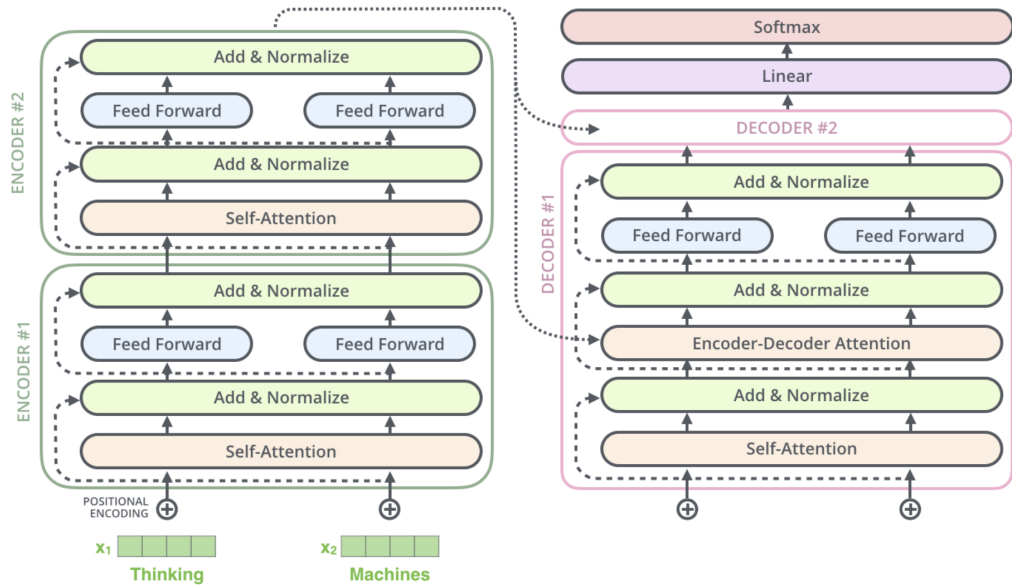


Рис.2.9. Подробная архитектура Transformer [3].

Здесь два кодировщика и декодировщика, а также присутствует слой Linear и Softmax — они нужны для преобразования выходного вектора декодера в распределение вероятностей по словарю целевого языка. Линейный слой проектирует вектор размерности d_{model} в вектор размерности, равной размеру целевого словаря. Softmax преобразует полученные значения в вероятности, сумма которых равна единице. Слово с максимальной вероятностью выбирается в качестве очередного слова перевода на каждом шаге генерации [4].

Для понимания работы каждого блока необходимо рассмотреть дополнительные компоненты, используемые после каждого слоя.

2.8 Полносвязная нейронная сеть

После каждого слоя внимания в архитектуре Transformer применяется полносвязная сеть, которая состоит из двух линейных преобразований с функцией активации ReLU между ними. Этот процесс можно представить в два этапа:

Сначала первый линейный слой преобразует входной вектор x размерности d_{model} в пространство размерности d_{ff} по формуле (2.5).

$$h = \text{ReLU}(xW_1 + b_1), \quad h \in \mathbb{R}^{d_{\text{ff}}}. \quad (2.6)$$

Затем второй линейный слой возвращает вектор к исходной размерности по формуле (2.6).

$$\text{FFN}(x) = hW_2 + b_2, \quad \text{FFN}(x) \in \mathbb{R}^{d_{\text{model}}}, \quad (2.7)$$

где $\text{ReLU}(z) = \max(0, z)$, $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ — матрицы весов, а $b_1 \in \mathbb{R}^{d_{\text{ff}}}$, $b_2 \in \mathbb{R}^{d_{\text{model}}}$ — векторы смещений.

В нашем случае $d_{\text{model}} = 512$ и $d_{\text{ff}} = 2048$ [4]. После применения функции активации ReLU второй линейный слой возвращает вектор к исходной размерности $d_{\text{model}} = 512$. Такая архитектура позволяет модели выявлять более сложные нелинейные взаимосвязи, расширяя представление признаков в более высокоразмерное пространство, а затем сжимая его обратно, рис. 2.10. Важно отметить, что полносвязная сеть применяется независимо к каждой позиции в последовательности.

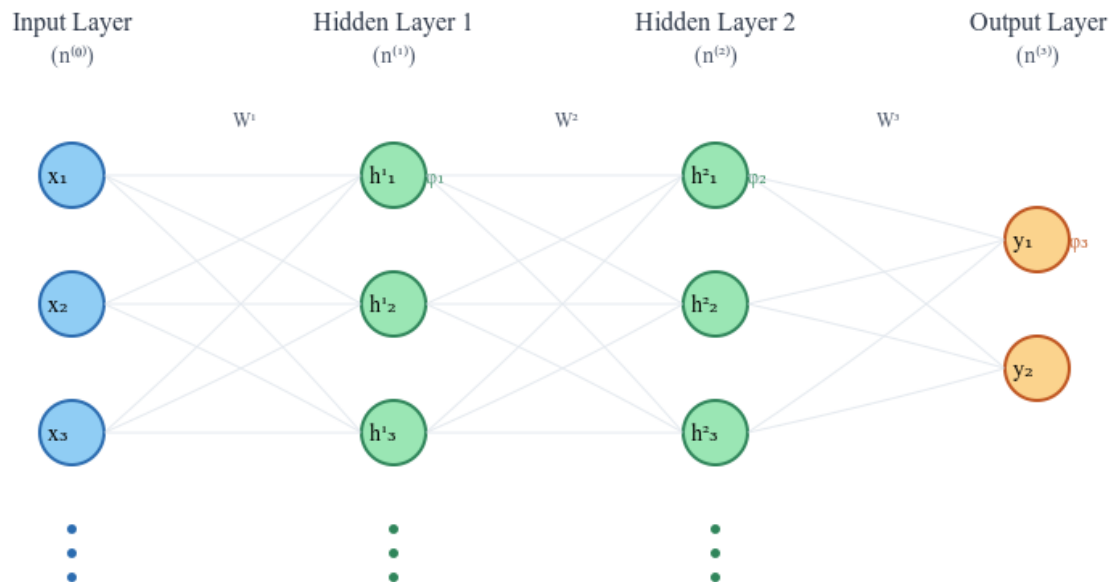


Рис.2.10. Полносвязная нейронная сеть (Feed-Forward Neural Network) [3].

2.9 Слои Linear и Softmax

После прохождения последовательности через стек декодеров на выходе формируется вектор признаков размерности d_{model} . Однако данный вектор ещё не соответствует конкретному слову целевого языка и требует преобразования.

Для этой цели используется линейный слой, который выполняет проекцию выходного вектора декодера в пространство размерности, равной объёму словаря целевого языка. В результате формируется вектор логитов, в котором каждому элементу соответствует одно слово словаря. Например, если словарь содержит 10 000 уникальных слов, то линейный слой сформирует вектор из 10 000 значений.

Полученные логиты сами по себе не являются вероятностями, поэтому далее применяется функция Softmax. Она преобразует значения логитов в распределение вероятностей, при котором все значения находятся в диапазоне от 0 до 1, а их сумма равна единице. Каждая вероятность отражает степень уверенности модели в выборе соответствующего слова.

На этапе генерации перевода в качестве следующего слова обычно выбирается токен, имеющий наибольшую вероятность. После этого выбранный токен передаётся на следующий шаг декодирования для формирования последующих слов предложения, как показано на рисунке 2.11.

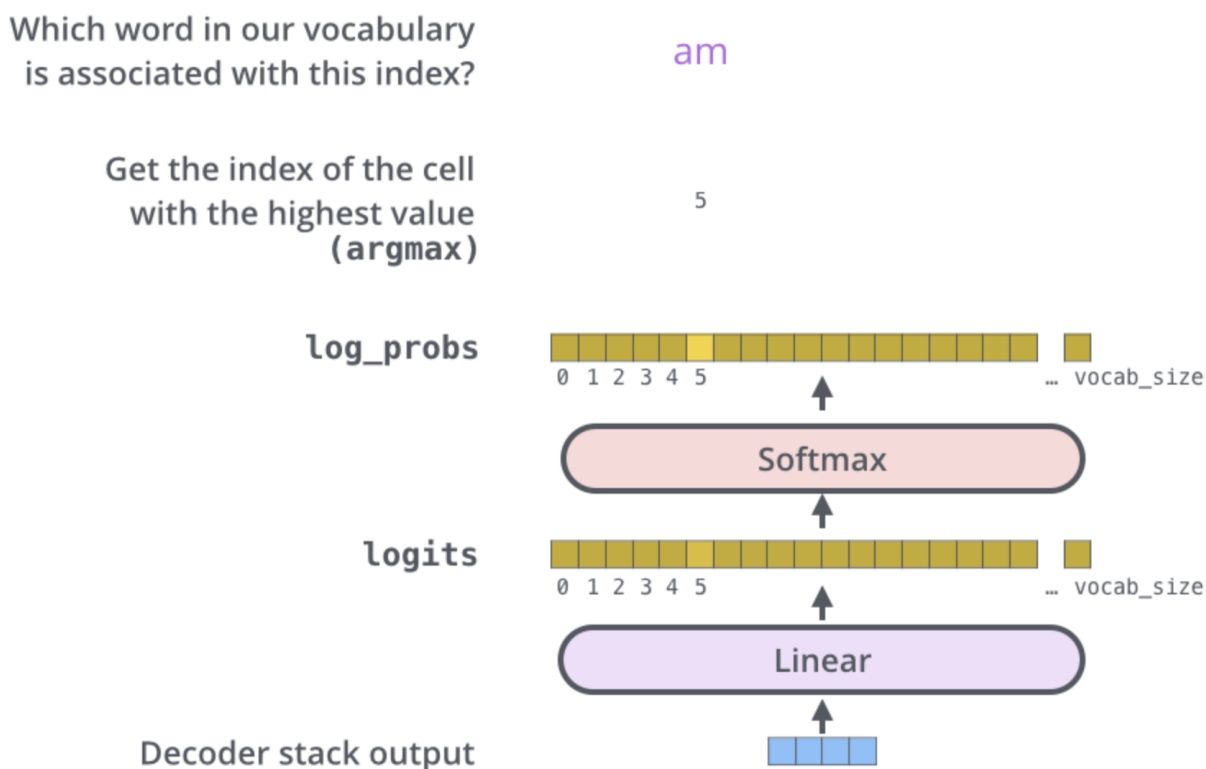


Рис.2.11. Слои Linear и Softmax [3].

Однако ключевым элементом архитектуры Transformer остаётся механизм внимания, определяющий способ формирования контекстных представлений слов.

2.10 Механизмы внимания

Одной из ключевых особенностей архитектуры Transformer является механизм внимания, благодаря которому модель может учитывать взаимосвязи между словами независимо от их положения в последовательности. В отличие от рекуррентных нейронных сетей, где информация передаётся последовательно от одного

слова к другому, механизм внимания позволяет одновременно анализировать всю входную последовательность. Это существенно повышает эффективность обработки длинных предложений и облегчает выявление как синтаксических, так и семантических зависимостей между словами.

Основой механизма внимания являются три вектора: запрос (Query, Q), ключ (Key, K) и значение (Value, V). Они вычисляются с помощью линейных преобразований входных представлений по формуле (2.8).

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (2.8)$$

где X — матрица входных эмбеддингов, а W_Q , W_K и W_V — обучаемые матрицы весов.

Далее вычисляется матрица оценок внимания как скалярное произведение матриц запросов и ключей по формуле (2.9).

$$QK^T. \quad (2.9)$$

Полученные значения масштабируются с целью предотвращения слишком больших значений при увеличении размерности пространства признаков по формуле (2.10).

$$\frac{QK^T}{\sqrt{d_k}}, \quad (2.10)$$

где d_k — размерность векторов ключей.

После масштабирования применяется функция Softmax, преобразующая оценки внимания в вероятностное распределение по формуле (2.11).

$$\text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right), \quad (2.11)$$

Итоговое представление вычисляется по формуле (2.12).

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (2.12)$$

Полученные коэффициенты внимания определяют степень влияния каждого слова последовательности на формирование нового представления текущего слова.

В архитектуре Transformer используются три разновидности механизма внимания: self-attention, multi-head attention и encoder-decoder attention. Несмотря

на использование общей формулы вычисления внимания, каждая из них выполняет собственную функцию.

2.10.1 Self-attention

Механизм self-attention применяется как в кодировщике, так и в декодере и позволяет каждому слову учитывать информацию обо всех остальных словах той же последовательности. Благодаря этому модель способна выявлять зависимости между словами независимо от расстояния между ними.

Особенностью данного механизма является то, что запросы, ключи и значения формируются из одной и той же последовательности по формуле (2.13).

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \quad (2.13)$$

В результате каждое слово получает новое представление, сформированное с учётом контекста всего предложения. Такой подход позволяет модели учитывать как близкие, так и удалённые зависимости между словами.

В кодировщике self-attention вычисляется без ограничений, тогда как в декодере используется маскирование (masked self-attention), запрещающее учитывать будущие токены последовательности. Это позволяет модели генерировать перевод последовательно, не используя информацию о ещё не сгенерированных словах.

2.10.2 Multi-head attention

Использование только одного механизма внимания ограничивает способность модели выявлять различные типы взаимосвязей между словами. Поэтому в архитектуре Transformer применяется механизм Multi-Head Attention, при котором вычисляются несколько механизмов внимания параллельно.

Для каждой головы внимания вычисляется собственный результат по формуле (2.14).

$$head_i = \text{Attention}(Q_i, K_i, V_i), \quad (2.14)$$

где $i = 1, \dots, h$, а h — количество голов внимания.

После вычисления всех голов их результаты объединяются операцией конкатенации и проходят через дополнительное линейное преобразование по формуле (2.15).

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O, \quad (2.15)$$

где W_O — обучаемая матрица весов.

Использование нескольких голов внимания позволяет модели одновременно анализировать различные аспекты входной последовательности. Одни головы могут выявлять синтаксические зависимости между словами, другие — семантические связи, что повышает качество формируемых представлений.

2.10.3 Encoder-decoder attention

Помимо механизма self-attention, в декодере используется механизм encoder-decoder attention. Его основная задача заключается в передаче информации от кодировщика к декодеру при генерации перевода.

В отличие от self-attention, запросы формируются выходом декодера, тогда как ключи и значения поступают из выхода кодировщика по формуле (2.16).

$$Q = \text{Decoder}, \quad K = \text{Encoder}, \quad V = \text{Encoder}. \quad (2.16)$$

После формирования запросов, ключей и значений вычисление внимания выполняется по формуле (2.12).

Благодаря этому механизму декодер определяет, на какие части исходного предложения следует обратить наибольшее внимание при генерации очередного слова перевода. Это обеспечивает согласованность между входной и выходной последовательностями и позволяет формировать более точный перевод.

2.11 Выводы

В данной главе были рассмотрены основные компоненты архитектуры Transformer: механизм внимания, многоголовое внимание, остаточные связи, нормализация слоёв, полносвязная нейронная сеть и выходные слои Linear и Softmax, изучены этапы подготовки входных данных.

ГЛАВА 3. ПРОЕКТИРОВАНИЕ, РЕАЛИЗАЦИЯ И ОБУЧЕНИЕ МОДЕЛИ TRANSFORMER

Глава посвящена практической реализации модели машинного перевода. Рассматриваются подготовка датасета OPUS-100 и предобработка данных (параграф 3.12), а также реализация архитектуры Transformer на PyTorch (параграф 3.13). В параграфе 3.16 приведены выводы по главе.

3.12 Подготовка корпуса OPUS-100 и предобработка данных

В качестве обучающего корпуса был выбран датасет OPUS-100 для пары языков «русский-английский», загруженный через библиотеку `datasets`. Датасет содержит 1 миллион тренировочных, 2000 валидационных и 2000 тестовых пар предложений. Код для его загрузки представлен на рисунке 3.12.

```
from datasets import load_dataset

dataset = load_dataset("Helsinki-NLP/opus-100", "en-ru")
```

Рис.3.12. Загрузка датасета OPUS-100.

Поскольку в датасете обнаружались артефакты в виде китайских иероглифов и символов индийского алфавита, перед обучением токенизатора данные фильтруются так, чтобы остались только латиница и кириллица. Для фильтрации таких примеров была реализована функция `is_clean`, представленная на рисунке 3.13, которая с помощью регулярных выражений пропускает только текст, состоящий из латиницы, кириллицы, цифр и базовых знаков препинания. Тексты нарезаются на батчи, на которых и обучается токенизатор, это представлено на рисунке П1.2.

```
import re

allowed_chars = re.compile(r"^[A-Za-zА-Яа-яЁё0-9\s.,!?:;()'\-\"«»-]*$")

def is_clean(text):
    return bool(allowed_chars.match(text))
```

Рис.3.13. Функция `is_clean` для фильтрации артефактов в датасете.

Токенизация выполнена с использованием алгоритма Byte Pair Encoding из библиотеки `tokenizers`. Словарь содержит 32 000 токенов, включая специальные токены: `[PAD]` для выравнивания последовательностей, `[UNK]` для неизвестных

слов, [BOS] и [EOS] для маркеров начала и конца предложения, это представлено на рисунке П1.1.

На этапе подготовки числовых данных предложения обрезаются до 200 токенов. К русским входным последовательностям добавляется токен [EOS], к английским целевым — токены [BOS] и [EOS]. Поскольку предложения внутри батча имеют разную длину и не могут быть напрямую объединены в один тензор, реализован класс `DataCollator`, который динамически дополняет короткие предложения токенами [PAD] до длины самого длинного предложения в конкретном батче, это представлено на рисунке 3.14. Токенизированный датасет оборачивается в `DataLoader`, отвечающий за перемешивание и нарезку данных на батчи. Из-за добавленных токенов [PAD] в модель дополнительно передаются маски выравнивания `src_padding_mask` и `tgt_padding_mask`, они указывают на позиции с [PAD]-токенами, а маска `generate_square_subsequent_mask` блокирует доступ декодера к будущим токенам. Загрузка данных выполняется через `DataLoader` с размером батча 4, с перемешиванием для тренировочной выборки, это представлено на рисунке П1.3

```
import torch

class DataCollator:
    def __init__(self, pad_id):
        self.pad_id = pad_id

    def __call__(self, batch):
        # Извлекаем списки ID из батча
        input_ids = [item['input_ids'] for item in batch]
        labels = [item['labels'] for item in batch]

        # Находим максимальные длины в этом конкретном батче
        max_input_len = max(len(x) for x in input_ids)
        max_label_len = max(len(x) for x in labels)

        # Дополняем
        padded_inputs = [x + [self.pad_id] * (max_input_len - len(x)) for x in input_ids]
        padded_labels = [x + [self.pad_id] * (max_label_len - len(x)) for x in labels]

        return {
            'input_ids': torch.tensor(padded_inputs, dtype=torch.long),
            'labels': torch.tensor(padded_labels, dtype=torch.long)
        }

collator = DataCollator(pad_id=pad_id)
```

Рис.3.14. Класс `DataCollator` для динамического выравнивания последовательностей.

3.13 Реализация архитектуры Transformer на PyTorch

Архитектура Transformer реализована на PyTorch в соответствии с оригинальной спецификацией из статьи "Attention Is All You Need".

Позиционное кодирование добавлено через класс `PositionalEncoding`, это представлено на рисунке 3.15. Механизм многоголового внимания `MultiHeadAttention` реализован с разделением на `nhead = 8` голов, каждая размерности $d_k = d_{\text{model}}/\text{nhead}$. Для каждой головы вычисляется scaled dot-product attention, результаты конкатенируются и проецируются через линейный слой.

```
import torch
import torch.nn as nn
import math

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=1000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        # 10000^(2i/d_model)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term) # Четные индексы
        pe[:, 1::2] = torch.cos(position * div_term) # Нечетные индексы
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]
```

Рис.3.15. Позиционное кодирование.

Слой энкодера `EncoderLayer` включает многоголовое self-attention и полносвязную сеть `FeedForward` с размерностью скрытого слоя 2048, каждый компонент дополнен остаточной связью и `LayerNorm`. Слой декодера `DecoderLayer` содержит masked self-attention, cross-attention к выходу энкодера и полносвязную сеть с аналогичной структурой.

Итоговая модель `SimpleTransformer` объединяет слой эмбеддингов, позиционное кодирование, стек из 6 слоёв энкодера и 6 слоёв декодера, а также выходной линейный слой для проекции в размер словаря. Размерность модели d_{model} установлена в 512. Класс модели представлен на рисунке 3.16.

```

class SimpleTransformer(nn.Module):
    def __init__(self, vocab_size, d_model=512, nhead=8, num_layers=6, dim_feedforward=2048, max_len=1000):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model, max_len=max_len)

        # Создаем цепочки слоев через nn.ModuleList
        self.encoder_layers = nn.ModuleList([EncoderLayer(d_model, nhead, dim_feedforward) for _ in range(num_layers)])
        self.decoder_layers = nn.ModuleList([DecoderLayer(d_model, nhead, dim_feedforward) for _ in range(num_layers)])

        self.out_linear = nn.Linear(d_model, vocab_size)

    def forward(self, src, tgt, src_padding_mask, tgt_padding_mask, tgt_mask):
        # Проход через Энкодер
        memory = self.pos_encoder(self.embedding(src))
        for layer in self.encoder_layers:
            memory = layer(memory, src_padding_mask)

        # Проход через Декодер
        x = self.pos_encoder(self.embedding(tgt))
        for layer in self.decoder_layers:
            x = layer(x, memory, tgt_mask, tgt_padding_mask, src_padding_mask)

        return self.out_linear(x)

```

Рис.3.16. Итоговая модель Transformer.

Обучение ведётся с функцией потерь CrossEntropyLoss с игнорированием [PAD] -токенов. Оптимизатор — Adam. Для работы с ограниченной видеопамью в 16 гб применено накопление градиентов за 8 шагов, что эквивалентно эффективному батчу 32. Чекпоинты сохраняются каждые 2000 батчей; лучшая модель определяется по минимальному значению валидационного loss, листинг программного кода (П1.4).

3.14 Обучение модели

Гиперпараметры модели Transformer выбраны на основе классической архитектуры [4] с учетом ограничений видеокарты GPU T4 на 16 ГБ. Для защиты от переобучения использовалась регуляризация Dropout с коэффициентом 0,1. Гиперпараметры представлены на рисунке 3.17.

```

vocab_size = tokenizer.get_vocab_size()

# Гиперпараметры
D_MODEL = 512          # Размерность эмбеддингов
NHEAD = 8              # Количество голов внимания
NUM_LAYERS = 6          # Количество слоёв encoder/decoder
DIM_FEEDFORWARD = 2048 # Размерности сети feed-forward

# для экономии памяти
BATCH_SIZE = 4          # вместо 32
ACCUMULATION_STEPS = 8  # 4 * 8 = 32
LEARNING_RATE = 5e-5
EPOCHS = 3

```

Рис.3.17. Гиперпараметры модели и обучения.

При первых тестах с размером батча 32 и длиной текста до 10 000 токенов видеопамять переполнялась, появлялась ошибка CUDA Out of Memory. Для решения этой проблемы максимальная длина предложений была ограничена с 10000 до 1000 токенов, чтобы убрать слишком длинные фразы и снизить нагрузку на память. Также физический размер батча был уменьшен до 4, но при этом была включена аккумуляция градиентов на 8 шагов. Это позволило обновлять веса модели раз в 8 шагов и сохранить нужный размер батча $4 \times 8 = 32$.

Обучение в течение одной эпохи заняло около 4 часов. Использовался оптимизатор Adam со скоростью обучения $\eta = 5 \times 10^{-5}$.

Качество перевода оценивается по метрикам BLEU, METEOR и TER на валидационной выборке. Метрика TER реализована через расстояние Левенштейна на уровне слов. Дополнительно реализована функция анализа ошибок, которая сопоставляет исходный текст, эталонный перевод и перевод модели на нескольких примерах.

3.15 Процесс генерации перевода

Входное предложение токенизируется, дополняется токеном [EOS] и подается в энкодер, который формирует закодированное представление исходного текста. Декодер инициализируется токеном [BOS] и на каждом шаге вычисляет следующий токен на основе уже сгенерированной части и выходных данных энкодера. На каждом шаге выбирается токен с максимальной вероятностью. Генерация останавливается при получении токена [EOS] или достижении максимальной длины последовательности в 100 токенов. После завершения генерации из последова-

тельности удаляется начальный токен [BOS], и оставшиеся токены декодируются в текст с помощью токенизатора, как показано в листинге программного кода (П1.5). Генерация перевода выполняется авторегрессивно, код для генерации представлен на рисунке 3.18, пример генерации на рисунке 3.18.

```
model = load_model("checkpoints/best_model.pt")

test_examples = [
    ("Привет, как дела?", "Hello, how are you?"),
    ("Я люблю машинное обучение", "I love machine learning"),
    ("Сегодня хорошая погода", "Today is good weather"),
    ("Это очень интересная задача", "This is a very interesting task"),
    ("Модель Transformer работает хорошо", "The Transformer model works well"),
]

translate_examples(model, test_examples)
```

Рис.3.18. Функция `translate_examples` для демонстрации перевода на тестовых примерах.

```
Пример 1:
RU: Привет, как дела?
EN (ожидаемый): Hello, how are you?
EN (перевод):   Hey , how are you ?
Пример 2:
RU: Я люблю машинное обучение
EN (ожидаемый): I love machine learning
EN (перевод):   I love a free school
Пример 3:
RU: Сегодня хорошая погода
EN (ожидаемый): Today is good weather
EN (перевод):   Tonight today ,
Пример 4:
RU: Это очень интересная задача
EN (ожидаемый): This is a very interesting task
EN (перевод):   It ' s a very interesting task
Пример 5:
RU: Модель Transformer работает хорошо
EN (ожидаемый): The Transformer model works well
EN (перевод):   Model – The – school works
```

Рис.3.19. Примеры перевода.

3.16 Выводы

В рамках практической части реализована архитектура Transformer. Подготовлен датасет OPUS-100, обучен BPE-токенизатор, реализованы все компоненты архитектуры, модель обучена и оценена по метрикам BLEU, METEOR и TER на валидационной выборке.

ГЛАВА 4. АНАЛИЗ КАЧЕСТВА ПЕРЕВОДА ОБУЧЕННОЙ МОДЕЛИ

Глава посвящена тестированию разработанной модели. Рассматриваются обучение модели (параграф 3.14), а также оценка качества перевода по метрикам BLEU, METEOR, TER и анализ ошибок (параграф 4.17). В параграфе 4.18 приведены выводы по главе.

4.17 Оценка качества перевода

Для оценки качества перевода обученной модели были использованы три стандартные метрики: BLEU (Bilingual Evaluation Understudy) [6], METEOR (Metric for Evaluation of Translation with Explicit Ordering) [5] и TER (Translation Edit Rate) [2]. Оценка проводилась на тестовой выборке из 2000 предложений. Результаты представлены в таблице 4.2.

Таблица 4.2

Результаты оценки качества перевода

Метрика	Значение
BLEU	20,01
METEOR	0,3927
TER	1,0747

Полученный показатель BLEU равен 20,01 при эталонном диапазоне от 0 до 100. Значения выше 20 считаются минимально достаточными для сохранения общего смысла предложений. Метрика METEOR составила 0,3927 при эталонном интервале от 0 до 1, где диапазон 0,3–0,5 соответствует среднему качеству перевода. По метрике TER оптимальным результатом является стремление к 0, а нормой для качественного перевода — диапазон 0,4–0,6. Итоговый показатель TER составил 1,0747. Превышение единицы обусловлено ошибкой генерации — избыточным циклическим повтором токенов.

Из таблицы 4.3 видно, что модель часто неправильно выбирает значение слов в контексте. Например, в первом примере фраза “вы просто не представляете” переводится как “you just don’t have to do it” вместо “you’ve no idea”; в третьем примере “обрушится” ошибочно переводится как “joke” вместо “collapse”.

Модель склонна к повторению слов, особенно в длинных предложениях. Примеры: “enough enough”, “school school”, “the amount of the amount”, “the home

Таблица 4.3

Анализ перевода

Исходное предложение	Эталонный перевод	Перевод модели
Было так холодно на улице, вы просто не представляете.	It's so cold out there, you've no idea.	It was so cold, you just don't have to do it.
Я всё знаю, мистер Уолкер.	Nothing gets by me, Mr. Walker.	I know everything, Mr. Walker.
Есть ли шанс, что все это просто обрушится?	Isn't there a chance this lot's just gonna collapse?	Is there anything that's just a joke?
Но два ученика — это не достаточно, чтобы держать открытой школу.	Two students are not enough to sustain a school.	But two — is not enough enough to keep the school school.
Они знали о деньгах, и они знали точную сумму.	They knew about the money and they knew the exact amount.	They knew about money, and they knew the amount of the amount.
Казалось, будто с ними легче разговаривать, чем с местными жителями, американцами, которые обслуживали нас и подавали нам еду.	It seemed they were probably easier to talk to than the local Americans who were waiting on us as and serving food.	It was like it was easier to talk to them than the Americans that had the Americans that had taken us and had brought us.
— Мы будем делать домашние задания.	— We'll do homework.	— We're gonna do the home of the home.

of the home”, “the Americans that had the Americans”. Это указывает на проблему с механизмом генерации, который не штрафует повторяющиеся токены.

Также ВРЕ-токенизатор разбивает слова на субтокены, что приводит к появлению пробелов внутри слов в исходных текстах. В примерах изначально было: “представля ете”, “Уол кер”, “обру шится”, “Каза лось”, “жи телями”, “американ цами”, “обслужи вали”, “домаш ние зада ния”. Эта проблема связана с используемым претокенизатором Whitespace и отсутствием специальных маркеров для склеивания субтокенов при декодировании.

4.18 Выводы

Валидационный loss составил 3,2843. Получены метрики качества перевода: BLEU — 20,01, METEOR — 0,3927, TER — 1,0747.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы цель исследования была успешно достигнута, а все поставленные задачи полностью решены. Была реализована и экспериментально исследована архитектура Transformer для задачи русско-английского машинного перевода.

В процессе работы были получены следующие результаты:

- А. Изучена архитектура Transformer, включая механизмы самовнимания, многоголовое внимание, остаточные связи и позиционное кодирование;
- В. Выполнена подготовка датасета OPUS-100: проведена фильтрация данных, обучен BPE-токенизатор с размером словаря 32 000 токенов, реализован пайплайн предобработки;
- С. Реализована архитектура Transformer на фреймворке PyTorch с размерностью $d_{model} = 512$, 6 слоями энкодера и декодера;
- Д. Проведено обучение модели на 1 миллионе параллельных предложений. Значение функции потерь на валидационной выборке составило 3,2843;
- Е. Проведена оценка качества перевода по общепринятым метрикам: BLEU — 20,01, METEOR — 0,3927, TER — 1,0747;
- Ф. Выполнен анализ ошибок генерации, выявивший семантические искажения, повторения слов и артефакты токенизации.

В ходе работы сформулирован ряд рекомендаций по дальнейшему совершенствованию модели, включающих замену алгоритма токенизации и введение механизма штрафа за повторение токенов.

Полученные результаты подтверждают работоспособность реализованной модели и могут быть применены при проектировании диалоговых ассистентов и специализированных систем машинного перевода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ru-en-transformer-nmt: открытый репозиторий исходного кода проекта. — 2026. — URL: <https://github.com/kirimusha/ru-en-transformer-nmt> (дата обращения: 14.07.2026).
2. A Study of Translation Edit Rate with Targeted Human Annotation / M. Snover [и др.] // Proceedings of the 7th Conference of the Association for Machine Translation in the Americas (AMTA). — 2006. — С. 223—231. — URL: <https://aclanthology.org/2006.amta-papers.16/>.
3. *Alammar J.* The Illustrated Transformer. — 2018. — URL: <https://jalammar.github.io/illustrated-transformer/> (дата обращения: 28.06.2026).
4. Attention is All You Need / A. Vaswani [и др.] // Advances in Neural Information Processing Systems. Т. 30. — Curran Associates, Inc., 2017. — С. 5998—6008. — URL: <http://arxiv.org/abs/1706.03762>.
5. *Banerjee S., Lavie A.* METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments // Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization. — 2005. — С. 65—72. — URL: <https://aclanthology.org/W05-0909/>.
6. BLEU: a Method for Automatic Evaluation of Machine Translation / К. Papineni [и др.] // Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL). — 2002. — С. 311—318. — URL: <https://aclanthology.org/P02-1040/>.
7. Is ChatGPT A Good Translator? Yes With GPT-4 As The Engine / W. Jiao [и др.]. — 2023. — URL: <https://arxiv.org/abs/2301.08745> (дата обращения: 11.07.2026).
8. *Lingvanex.* Нейронный машинный перевод: что это? — 2023. — URL: <https://lingvanex.com/ru/blog/what-is-neural-machine-translation/> (дата обращения: 28.06.2026).
9. *Sennrich R., Haddow B., Birch A.* Neural Machine Translation of Rare Words with Subword Units // Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL). — 2016. — С. 1715—1725. — URL: <https://arxiv.org/abs/1508.07909>.
10. *Technolady.* История машинного перевода: от Декарта до нейросетей. — 2021. — URL: <https://www.securitylab.ru/blog/personal/Technolady/355284.php> (дата обращения: 28.06.2026).

11. The Mathematics of Statistical Machine Translation: Parameter Estimation / P. F. Brown [и др.] // Computational Linguistics. — 1993. — Т. 19, № 2. — С. 263—311. — URL: <https://aclanthology.org/J93-2003/>.
12. Бюро переводов Б2Б. История машинного перевода: развитие и появление. — 2023. — URL: <https://b2bperevod.ru/articles/tekhnologii-perevoda/istoriya-mashinnogo-perevoda/> (дата обращения: 28.06.2026).
13. Зотов А. Машинный перевод. Как развивалась технология. — 2024. — URL: <https://habr.com/ru/articles/1003076/> (дата обращения: 28.06.2026).
14. Иванов И. И., Петров П. П. Обзор методов глубокого обучения для нейронного машинного перевода // Научно-технический вестник. — 2020. — URL: <http://www.nauteh-journal.ru/files/e986ec4f-45ec-4d6d-9dff-6541c7a0449f> (дата обращения: 28.06.2026).
15. Кафедра лингвистики МГУ. Системы машинного перевода. — 2019. — URL: <http://lingva.fl.msu.ru/2019/08/%D1%81%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D1%8B-%D0%BC%D0%B0%D1%88%D0%B8%D0%BD%D0%BD%D0%BE%D0%B3%D0%BE-%D0%BF%D0%B5%D1%80%D0%B5%D0%B2%D0%BE%D0%B4%D0%B0/> (дата обращения: 28.06.2026).
16. Козлов В. А. Проектирование сервиса машинного перевода с использованием современных NMT архитектур // Электронная библиотека БГУ. — 2022. — URL: <https://elib.bsu.by/bitstream/123456789/328340/1/395-399.pdf> (дата обращения: 28.06.2026).
17. Радиолоцман. Что такое нейронный машинный перевод (NMT): история и виды. — 2022. — URL: <https://www.radiolocman.com/press-rel/rel.html?di=429-evolyuciya-perevoda-ot-pervyh-mashin-k-neyronnym-setyam> (дата обращения: 28.06.2026).
18. Семенов С. С. Обзор архитектуры рекуррентного Трансформера в контексте задач NLP // Труды МФТИ. — 2021. — Т. 64, № 1. — URL: https://mipt.ru/upload/%D0%A2%D1%80%D1%83%D0%B4%D1%8B_%D0%BC%D1%84%D1%82%D0%B8/64/01.pdf (дата обращения: 28.06.2026).

Приложение 1

Листинг программного кода

Полный исходный код в виде Jupyter-ноутбука также доступен в репозитории [1].

Ниже представлен полный код, использованный в данной работе, разделённый на файлы по функциональному назначению: подготовка данных и токенизация, реализация архитектуры модели, обучение, инференс и оценка качества перевода.

П1.1 Загрузка датасета и обучение токенизатора

Листинг П1.1

tokenizer.py

```

!pip install datasets

from datasets import load_dataset
5
dataset = load_dataset("Helsinki-NLP/opus-100", "en-ru")

print(dataset)

10 print(dataset['train'][:2])

from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
15 from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
tokenizer.pre_tokenizer = Whitespace() # нарезаем по пробелам
20

trainer = BpeTrainer(
    special_tokens=["[PAD]", "[UNK]", "[BOS]", "[EOS]"], # [
        PAD] - это пустышки, использую для выравнивания
    vocab_size=32000 # это базовый размер, который использовал
        и авторы статьи "Attention is all you need" для англо-ф
        ранцузского перевода
)
25

```

```

import re

allowed_chars = re.compile(r"^[A-Za-zА-Яа-яЁё0-9\s.,!?:;()
    '\-\' " ]*$")

30
def is_clean(text):
    return bool(allowed_chars.match(text))

def batch_iterator(batch_size=1000):
35
    for i in range(0, len(dataset["train"]), batch_size):
        batch = dataset["train"][i : i + batch_size]
        text_batch = []
        for item in batch["translation"]:
            if is_clean(item["en"]) and is_clean(item["ru"]):
40
                text_batch.append(item["en"])
                text_batch.append(item["ru"])
        yield text_batch

tokenizer.train_from_iterator(batch_iterator(), trainer)
45
# tokenizer.save("opus100_bpe_tokenizer.json")

```

П1.2 Подготовка числовых данных

Листинг П1.2

data_preparation.py

```

# ID специальных токенов из токенизатора
pad_id = tokenizer.token_to_id("[PAD]")
bos_id = tokenizer.token_to_id("[BOS]")
5 eos_id = tokenizer.token_to_id("[EOS]")

MAX_LENGTH = 200

def preprocess_function(examples):
10
    input_ids = []
    labels = []

    for item in examples["translation"]:
        # Токенизируем тексты и обрезаем их
15
        ru_encoded = tokenizer.encode(item["ru"]).ids[:
            MAX_LENGTH]
        en_encoded = tokenizer.encode(item["en"]).ids[:
            MAX_LENGTH]

```

```

20     # Вход энкодера (русский)
        ru_sequence = ru_encoded + [eos_id] # текст + [EOS]

        # Выход декодера (английский)
        en_sequence = [bos_id] + en_encoded + [eos_id] # [BOS]
            + текст + [EOS]

        input_ids.append(ru_sequence)
25     labels.append(en_sequence)

    \# 'input_ids' пойдут в энкодер, а 'labels' в декодер
      в качестве таргета
    return {"input_ids": input_ids, "labels": labels}

30 tokenized_dataset = dataset.map(
    preprocess_function,
    batched=True,
    remove_columns=["translation"]
)
35 print(tokenized_dataset)

print(tokenized_dataset['train'][:2])

40 import torch

class DataCollator:
    def __init__(self, pad_id):
        self.pad_id = pad_id
45

    def __call__(self, batch):
        # Извлекаем списки ID из батча
        input_ids = [item['input_ids'] for item in batch]
        labels = [item['labels'] for item in batch]

50

        # Находим максимальные длины в этом конкретном батче
        max_input_len = max(len(x) for x in input_ids)
        max_label_len = max(len(x) for x in labels)

55

        # Дополняем
        padded_inputs = [x + [self.pad_id] * (max_input_len -
            len(x)) for x in input_ids]
        padded_labels = [x + [self.pad_id] * (max_label_len -
            len(x)) for x in labels]

```

```

        return {
            'input_ids': torch.tensor(padded_inputs, dtype=
60         torch.long),
            'labels': torch.tensor(padded_labels, dtype=torch.
                long)
        }

65 collator = DataCollator(pad_id=pad_id)

    from torch.utils.data import DataLoader

70 BATCH_SIZE = 4

    train_dataloader = DataLoader(
        tokenized_dataset["train"],
        batch_size=BATCH_SIZE,
75         shuffle=True,
        collate_fn=collator
    )

    # Для валидации перемешивание не требуется
80 val_dataloader = DataLoader(
        tokenized_dataset["validation"],
        batch_size=BATCH_SIZE,
        shuffle=False,
        collate_fn=collator
85 )

    # проверка
    first_batch = next(iter(train_dataloader))
    print("Размерность входных данных (RU):", first_batch['
        input_ids'].shape)
90 print("Размерность целевых данных (EN):", first_batch['labels'
    ].shape)

    # Маска указывает True там, где стоит [PAD], чтобы attention и
        игнорировал эти позиции
    src_key_padding_mask = (first_batch['input_ids'] == pad_id)
    tgt_key_padding_mask = (first_batch['labels'] == pad_id)

```

П1.3 Код модели Transformer

transformer_model.py

```

def generate_square_subsequent_mask(sz, device):
    \# Создаем матрицу размера [sz, sz] из единиц, берем верхн
        ий треугольник (без главной диагонали)
    mask = (torch.triu(torch.ones((sz, sz), device=device)) ==
        1).transpose(0, 1)
5    \# Преобразуем в float mask: 0.0      можно смотреть, -inf
        нельзя смотреть
    mask = mask.float().masked_fill(mask == 0, float('-inf')).
        masked_fill(mask == 1, float(0.0))
    return mask

import torch
10 import torch.nn as nn
import math

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=1000):
15         super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float)
            .unsqueeze(1)
        \# Формула из статьи:  $10000^{-(2i/d\_model)}$ 
        div_term = torch.exp(torch.arange(0, d_model, 2).float
            () * (-math.log(10000.0) / d_model))

20         pe[:, 0::2] = torch.sin(position * div_term) \# Четные
            индексы
        pe[:, 1::2] = torch.cos(position * div_term) \# Нечетны
            е индексы
        self.register_buffer('pe', pe.unsqueeze(0))

25     def forward(self, x):
        \# x: [batch_size, seq_len, d_model]
        return x + self.pe[:, :x.size(1)]

class MultiHeadAttention(nn.Module):
30     def __init__(self, d_model, nhead):
        super().__init__()
        self.nhead = nhead
        self.d_k = d_model // nhead

```

```

35     # Проекции для получения Q, K, V и финального объединения
        # O
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)

40
def forward(self, q, k, v, mask=None, key_padding_mask=
    None):
    batch_size = q.size(0)

    # Линейный слой + разделение на головы
    # [batch_size, seq_len, d_model] в [batch_size, nhead,
    #   seq_len, d_k]
    Q = self.w_q(q).view(batch_size, -1, self.nhead, self.
        d_k).transpose(1, 2)
    K = self.w_k(k).view(batch_size, -1, self.nhead, self.
        d_k).transpose(1, 2)
    V = self.w_v(v).view(batch_size, -1, self.nhead, self.
        d_k).transpose(1, 2)

50
    # Scaled Dot-Product Attention (Формула из статьи)
    # scores: [batch_size, nhead, seq_len_q, seq_len_k]
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.
        sqrt(self.d_k)

    # Применяем каузальную маску (для декодера)
55
    if mask is not None:
        scores = scores + mask.unsqueeze(0).unsqueeze(1)

    # Применяем padding маску (скрываем [PAD] токены)
    if key_padding_mask is not None:
60
        scores = scores.masked_fill(key_padding_mask.
            unsqueeze(1).unsqueeze(2), float('-inf'))

    # Softmax + умножение на V
    attn_weights = torch.softmax(scores, dim=-1)
    context = torch.matmul(attn_weights, V)

65
    # Склеиваем головы обратно (Concat)
    context = context.transpose(1, 2).contiguous().view(
        batch_size, -1, self.nhead * self.d_k)
    return self.w_o(context)

70 class FeedForward(nn.Module):

```

```

def __init__(self, d_model, dim_feedforward):
    super().__init__()
    self.linear1 = nn.Linear(d_model, dim_feedforward)
    self.relu = nn.ReLU()
75     self.linear2 = nn.Linear(dim_feedforward, d_model)

def forward(self, x):
    return self.linear2(self.relu(self.linear1(x)))

80 class EncoderLayer(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, nhead)
        self.ff = FeedForward(d_model, dim_feedforward)
85     self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)

    def forward(self, src, src_padding_mask):
        # Attention + Residual + Norm
90     attn_out = self.self_attn(src, src, src,
                                key_padding_mask=src_padding_mask)
        x = self.norm1(src + attn_out)

        # FeedForward + Residual + Norm
95     return self.norm2(x + self.ff(x))

class DecoderLayer(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward):
        super().__init__()
100    self.self_attn = MultiHeadAttention(d_model, nhead)
        self.cross_attn = MultiHeadAttention(d_model, nhead)
        self.ff = FeedForward(d_model, dim_feedforward)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
105    self.norm3 = nn.LayerNorm(d_model)

    def forward(self, tgt, memory, tgt_mask, tgt_padding_mask,
memory_padding_mask):
        # Masked Self-Attention
        attn1 = self.self_attn(tgt, tgt, tgt, mask=tgt_mask,
                                key_padding_mask=tgt_padding_mask)
110    x = self.norm1(tgt + attn1)

```

```

115     # Encoder-Decoder Cross-Attention (Query берем из x, a
        Key u Value из памяти кодера)
        attn2 = self.cross_attn(x, memory, memory,
                                key_padding_mask=memory_padding_mask)
        x = self.norm2(x + attn2)

        # FeedForward
        return self.norm3(x + self.ff(x))

class SimpleTransformer(nn.Module):
120     def __init__(self, vocab_size, d_model=512, nhead=8,
        num_layers=6, dim_feedforward=2048, max_len=1000):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model, max_len
                                              =max_len)

125     # Создаем цепочки слоев через nn.ModuleList
        self.encoder_layers = nn.ModuleList([EncoderLayer(
            d_model, nhead, dim_feedforward) for _ in range(
                num_layers)])
        self.decoder_layers = nn.ModuleList([DecoderLayer(
            d_model, nhead, dim_feedforward) for _ in range(
                num_layers)])

        self.out_linear = nn.Linear(d_model, vocab_size)

130     def forward(self, src, tgt, src_padding_mask,
        tgt_padding_mask, tgt_mask):
        # Проход через Энкодер
        memory = self.pos_encoder(self.embedding(src))
        for layer in self.encoder_layers:
135             memory = layer(memory, src_padding_mask)

        # Проход через Декoder
        x = self.pos_encoder(self.embedding(tgt))
        for layer in self.decoder_layers:
140             x = layer(x, memory, tgt_mask, tgt_padding_mask,
                src_padding_mask)

        return self.out_linear(x)

```

П1.4 Код обучения

train.py

```

vocab_size = tokenizer.get_vocab_size()

# Гиперпараметры
5 D_MODEL = 512          # Embedding dimension
  NHEAD = 8              # Number of attention heads
  NUM_LAYERS = 6          # Number of encoder/decoder layers
  DIM_FEEDFORWARD = 2048 # Dimension of the feed-forward network

10 # для экономии памяти
  BATCH_SIZE = 4          # вместо 32 (помещается в память)
  ACCUMULATION_STEPS = 8  # 4 * 8 = 32 ( batch size)
  LEARNING_RATE = 5e-5    # чуть меньше для стабильности
  EPOCHS = 3              # оставляем как было

15
  model = SimpleTransformer(
      vocab_size=vocab_size,
      d_model=D_MODEL,
      nhead=NHEAD,
20     num_layers=NUM_LAYERS,
      dim_feedforward=DIM_FEEDFORWARD,
      max_len=1000
  )

25 # Move the model to the GPU if available (you're using a T4
    GPU)
  device = torch.device("cuda" if torch.cuda.is_available() else
      "cpu")
  model = model.to(device)
  print(f"Model is running on: {device}")

30 import torch.optim as optim
  import gc
  import matplotlib.pyplot as plt
  import os
  from IPython.display import clear_output

35
  # папка для чекпоинтов
  CHECKPOINT_DIR = "checkpoints"
  os.makedirs(CHECKPOINT_DIR, exist_ok=True)

40 criterion = nn.CrossEntropyLoss(ignore_index=pad_id)
  optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

```

```

train_losses = []
val_losses = []
45 best_val_loss = float('inf')

def validate(model, val_dataloader, criterion, device):
    model.eval()
    total_loss = 0
50
    with torch.no_grad():
        for batch in val_dataloader:
            inputs = batch['input_ids'].to(device)
            labels = batch['labels'].to(device)
55

            tgt_input = labels[:, :-1]
            tgt_out = labels[:, 1:]

            src_padding_mask = (inputs == pad_id).to(device)
            tgt_padding_mask = (tgt_input == pad_id).to(device)
60
            )
            tgt_mask = generate_square_subsequent_mask(
                tgt_input.size(1), device)

            logits = model(inputs, tgt_input, src_padding_mask
                           , tgt_padding_mask, tgt_mask)
            loss = criterion(logits.view(-1, logits.size(-1)),
                             tgt_out.contiguous().view(-1))
65

            total_loss += loss.item()

    model.train()
    return total_loss / len(val_dataloader)
70

def save_checkpoint(model, optimizer, epoch, batch_idx, loss,
                    is_best=False):
    checkpoint = {
        'epoch': epoch,
        'batch': batch_idx,
75
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'loss': loss,
        'best_val_loss': best_val_loss,
        'train_losses': train_losses,
80
        'val_losses': val_losses,
    }

```

```

torch.save(checkpoint, os.path.join(CHECKPOINT_DIR, "
    checkpoint_latest.pt"))

85 if is_best:
    torch.save(model.state_dict(), os.path.join(
        CHECKPOINT_DIR, "best_model.pt"))
    print(f"Лучшая модель сохранена! Val Loss: {loss:.4f}"
        )

def plot_losses(train_losses, val_losses):
90 clear_output(wait=True) # Очищает вывод ячейки

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # График Train Loss
95 axes[0].plot(range(1, len(train_losses) + 1), train_losses
        , 'b-', linewidth=2, label='Train Loss')
    axes[0].set_xlabel('Эпоха')
    axes[0].set_ylabel('Loss')
    axes[0].set_title('Train Loss')
    axes[0].legend()
100 axes[0].grid(True, alpha=0.3)

    # График Validation Loss
    axes[1].plot(range(1, len(val_losses) + 1), val_losses, 'r
        -', linewidth=2, label='Validation Loss')
    axes[1].set_xlabel('Эпоха')
105 axes[1].set_ylabel('Loss')
    axes[1].set_title('Validation Loss')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    # Добавляем общий заголовок с лучшим loss
110 if val_losses:
        best_val = min(val_losses)
        best_epoch = val_losses.index(best_val) + 1
        fig.suptitle(f'Обучение модели | Лучший Val Loss: {
            best_val:.4f} (эпоха {best_epoch})', fontsize=14)
115

    plt.tight_layout()
    plt.show()

checkpoint_latest_path = os.path.join(CHECKPOINT_DIR, "
    checkpoint_latest.pt")

```

```

120 start_epoch = 0
    start_batch = 0

    if os.path.exists(checkpoint_latest_path):
        print("Найден чекпоинт! Восстанавливаю")
125     checkpoint = torch.load(checkpoint_latest_path)

        model.load_state_dict(checkpoint['model_state_dict'])
        optimizer.load_state_dict(checkpoint['optimizer_state_dict'])

130     start_epoch = checkpoint['epoch']
        start_batch = checkpoint['batch'] + 1
        best_val_loss = checkpoint.get('best_val_loss', float('inf'))
        train_losses = checkpoint.get('train_losses', [])
        val_losses = checkpoint.get('val_losses', [])

135     print(f"Восстановлено: эпоха {start_epoch}, батч {
        checkpoint['batch']}")

        # Показываем историю обучения
        if train_losses:
140             print(f"История: {len(train_losses)} эпох, лучший Val
                    Loss: {min(val_losses):.4f}")
            plot_losses(train_losses, val_losses)
        else:
            print("Начинаем обучение с нуля")

145 CHECKPOINT_INTERVAL = 2000

    model.train()
    for epoch in range(start_epoch, EPOCHS):
        total_loss = 0
150         optimizer.zero_grad()

        for batch_idx, batch in enumerate(train_dataloader):
            # Пропускаем уже пройденные батчи
            if epoch == start_epoch and batch_idx < start_batch:
155                 continue

            # Forward
            inputs = batch['input_ids'].to(device)
            labels = batch['labels'].to(device)
160

```

```

tgt_input = labels[:, :-1]
tgt_out = labels[:, 1:]

src_padding_mask = (inputs == pad_id).to(device)
165 tgt_padding_mask = (tgt_input == pad_id).to(device)
tgt_mask = generate_square_subsequent_mask(tgt_input.
    size(1), device)

logits = model(inputs, tgt_input, src_padding_mask,
    tgt_padding_mask, tgt_mask)
loss = criterion(logits.view(-1, logits.size(-1)),
    tgt_out.contiguous().view(-1))
170

loss = loss / ACCUMULATION_STEPS
loss.backward()

if (batch_idx + 1) % ACCUMULATION_STEPS == 0:
175     optimizer.step()
    optimizer.zero_grad()

total_loss += loss.item() * ACCUMULATION_STEPS

180 # Очистка памяти
if batch_idx % 50 == 0:
    torch.cuda.empty_cache()
    gc.collect()

185 # Чекпоинт
if batch_idx % CHECKPOINT_INTERVAL == 0 and batch_idx
    > 0:
    real_loss = loss.item() * ACCUMULATION_STEPS
    save_checkpoint(
        model, optimizer, epoch, batch_idx, real_loss,
190     is_best=False
    )
    print(f"Чекпоинт сохранен: эпоха {epoch}, батч {
        batch_idx}, Loss: {real_loss:.4f}")

# Логг
195 if batch_idx % 100 == 0:
    real_loss = loss.item() * ACCUMULATION_STEPS
    print(f"Эпоха [{epoch+1}/{EPOCHS}], Батч {
        batch_idx}, Loss: {real_loss:.4f}")

# конец эпохи

```

```

200     avg_train_loss = total_loss / len(train_dataloader)
        train_losses.append(avg_train_loss)

        # валидация
        print("\nВыполняется валидация")
205     avg_val_loss = validate(model, val_dataloader, criterion,
                               device)
        val_losses.append(avg_val_loss)

        # результаты
        print(f"Итог Эпохи {epoch+1}:")
210     print(f"    Train Loss: {avg_train_loss:.4f}")
        print(f"    Val Loss:    {avg_val_loss:.4f}")

        # сохраняем лучшую
215     if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            save_checkpoint(
                model, optimizer, epoch, batch_idx, avg_val_loss,
                is_best=True
220         )

        plot_losses(train_losses, val_losses)

        torch.save(model.state_dict(), os.path.join(CHECKPOINT_DIR
            , f"model_epoch_{epoch+1}.pt"))
225     print(f"Модель эпохи {epoch+1} сохранена как model_epoch_{
            epoch+1}.pt\n")

        plot_losses(train_losses, val_losses)

        print(f"\nОбучение завершено!")
230     print(f"    Лучший Val Loss: {best_val_loss:.4f}")

```

П1.5 Код процесса генерации текста

Листинг П1.5

inference.py

```

import torch
import torch.nn.functional as F

5
def load_model(model_path="checkpoints/best_model.pt"):

```

```

model = SimpleTransformer(
    vocab_size=vocab_size,
    d_model=D_MODEL,
10     nhead=NHEAD,
    num_layers=NUM_LAYERS,
    dim_feedforward=DIM_FEEDFORWARD,
    max_len=1000
).to(device)

15
model.load_state_dict(torch.load(model_path))
model.eval()
print(f"Модель загружена из {model_path}")
return model

20
def translate(model, text, max_len=100):
    model.eval()

    # Токенизируем входной текст
25     ru_encoded = tokenizer.encode(text).ids

    # Обрезаем, если слишком длинный
    if len(ru_encoded) > 500:
        ru_encoded = ru_encoded[:500]

30
    # Добавляем [EOS]
    src_tokens = ru_encoded + [eos_id]
    src_tensor = torch.tensor([src_tokens], dtype=torch.long).
        to(device)

    # Создаем маску для энкодера
35     src_padding_mask = (src_tensor == pad_id).to(device)

    # Прогоняем через энкодер
    with torch.no_grad():
40         memory = model.pos_encoder(model.embedding(src_tensor))
        for layer in model.encoder_layers:
            memory = layer(memory, src_padding_mask)

    # Генерация перевода (пошагово)
45     tgt_tokens = [bos_id] # начинаем с [BOS]

    for _ in range(max_len):
        tgt_tensor = torch.tensor([tgt_tokens], dtype=torch.
            long).to(device)

```

```

tgt_mask = generate_square_subsequent_mask(len(
    tgt_tokens), device)
tgt_padding_mask = (tgt_tensor == pad_id).to(device)

with torch.no_grad():
    x = model.pos_encoder(model.embedding(tgt_tensor))
    for layer in model.decoder_layers:
        x = layer(x, memory, tgt_mask,
            tgt_padding_mask, src_padding_mask)

    logits = model.out_linear(x[:, -1, :])

    # Берем токен с максимальной вероятностью (greedy)
    next_token = torch.argmax(logits, dim=-1).item()

    # Если сгенерирован [EOS] останавливаемся
    if next_token == eos_id:
        break

    tgt_tokens.append(next_token)

# Декодируем токены в текст
translated_text = tokenizer.decode(tgt_tokens[1:])
return translated_text

def translate_examples(model, examples):
    print("примеры перевода")
    print()

    for i, (ru_text, en_expected) in enumerate(examples, 1):
        print(f"Пример {i}:")
        print(f"  RU: {ru_text}")
        print(f"  EN (ожидаемый): {en_expected}")

        translated = translate(model, ru_text)
        print(f"  EN (перевод): {translated}")

def interactive_translate(model):
    print()
    print("интерактивный перевод")
    print()
    print("Введите текст на русском (или 'exit' для выхода):\n")

    while True:

```

```

    text = input("RU: ").strip()
    if text.lower() in ['exit', 'quit', 'выход']:
        print("До свидания!")
        break
95
    if not text:
        continue

    translated = translate(model, text)
100    print(f"EN: {translated}\n")

# Загружаем лучшую модель
model = load_model("checkpoints/best_model.pt")

105 # Примеры для теста
test_examples = [
    ("Привет, как дела?", "Hello, how are you?"),
    ("Я люблю машинное обучение", "I love machine learning"),
    ("Сегодня хорошая погода", "Today is good weather"),
110    ("Это очень интересная задача", "This is a very
        interesting task"),
    ("Модель Transformer работает хорошо", "The Transformer
        model works well"),
]

# Переводим примеры
115 translate_examples(model, test_examples)

# Интерактивный режим (раскомментируйте, если хотите)
# interactive_translate(model)

```

П1.6 Код оценки модели

Листинг П1.6

evaluate.py

```

!pip install sacrebleu torchmetrics

import nltk
5 nltk.download('wordnet')
  nltk.download('omw-1.4')

!pip install sacrebleu

10 import sacrebleu

```

```

from nltk.translate.meteor_score import meteor_score

def calculate_ter(hypotheses, references):
    total_edits = 0
15    total_words = 0

    for hyp, ref in zip(hypotheses, references):
        hyp_words = hyp.split()
        ref_words = ref.split()

20        # Простейшая реализация TER через расстояние Левенштейн
        на на уровне слов
        n = len(hyp_words)
        m = len(ref_words)

25        dp = [[0] * (m + 1) for _ in range(n + 1)]
        for i in range(n + 1):
            dp[i][0] = i
        for j in range(m + 1):
            dp[0][j] = j

30        for i in range(1, n + 1):
            for j in range(1, m + 1):
                if hyp_words[i-1] == ref_words[j-1]:
                    dp[i][j] = dp[i-1][j-1]
                else:
35                    dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1],
                                         dp[i-1][j-1])

        total_edits += dp[n][m]
        total_words += m

40        return total_edits / total_words if total_words > 0 else 0

def calculate_metrics(model, dataloader, max_samples=100):
    model.eval()

45    references = []
    hypotheses = []

    with torch.no_grad():
50        for batch_idx, batch in enumerate(dataloader):
            if batch_idx >= max_samples:
                break

```

```

src = batch['input_ids'][0:1].to(device)
tgt = batch['labels'][0:1].to(device)

memory = model.pos_encoder(model.embedding(src))
for layer in model.encoder_layers:
    memory = layer(memory, (src == pad_id).to(
        device))

tgt_tokens = [bos_id]

for _ in range(100):
    tgt_tensor = torch.tensor([tgt_tokens], dtype=
        torch.long).to(device)
    tgt_mask = generate_square_subsequent_mask(len
        (tgt_tokens), device)
    tgt_padding_mask = (tgt_tensor == pad_id).to(
        device)
    src_padding_mask = (src == pad_id).to(device)

    x = model.pos_encoder(model.embedding(
        tgt_tensor))
    for layer in model.decoder_layers:
        x = layer(x, memory, tgt_mask,
            tgt_padding_mask, src_padding_mask)

    logits = model.out_linear(x[:, -1, :])
    next_token = torch.argmax(logits, dim=-1).item
        ()

    if next_token == eos_id:
        break
    tgt_tokens.append(next_token)

ref_text = tokenizer.decode(tgt[0].tolist())
ref_text = ref_text.replace('[BOS]', '').replace('
    [EOS]', '').replace('[PAD]', '').strip()

hyp_text = tokenizer.decode(tgt_tokens[1:])

references.append(ref_text)
hypotheses.append(hyp_text)

bleu = sacrebleu.corpus_bleu(hypotheses, [[ref] for ref in
    references])

```

```

90     meteor_scores = []
    for hyp, ref in zip(hypotheses, references):
        score = meteor_score([ref.split()], hyp.split())
        meteor_scores.append(score)
    meteor = sum(meteor_scores) / len(meteor_scores)

95

    ter = calculate_ter(hypotheses, references)

    print(f"BLEU: {bleu.score:.2f}")
    print(f"METEOR: {meteor:.4f}")
100    print(f"TER: {ter:.4f}")

    return bleu.score, meteor, ter

metrics = calculate_metrics(model, val_dataloader, max_samples
                             =100)

105
import pandas as pd

results = {
    'Метрика': ['BLEU', 'METEOR', 'TER'],
110    'Значение': [metrics[0], metrics[1], metrics[2]]
}
df = pd.DataFrame(results)
print(df.to_string(index=False))

115 def analyze_errors(model, dataloader, num_examples=10):
    model.eval()

    errors = []

120
    with torch.no_grad():
        for batch_idx, batch in enumerate(dataloader):
            if batch_idx >= num_examples:
                break

125
            src = batch['input_ids'][0:1].to(device)
            tgt = batch['labels'][0:1].to(device)

            src_text = tokenizer.decode(src[0].tolist())
            src_text = src_text.replace('[BOS]', '').replace('
[EOS]', '').replace('[PAD]', '').strip()

130
            ref_text = tokenizer.decode(tgt[0].tolist())

```

```

ref_text = ref_text.replace('[BOS]', '').replace('
[EOS]', '').replace('[PAD]', '').strip()

memory = model.pos_encoder(model.embedding(src))
135 for layer in model.encoder_layers:
    memory = layer(memory, (src == pad_id).to(
        device))

tgt_tokens = [bos_id]
140 for _ in range(100):
    tgt_tensor = torch.tensor([tgt_tokens], dtype=
        torch.long).to(device)
    tgt_mask = generate_square_subsequent_mask(len
        (tgt_tokens), device)
    tgt_padding_mask = (tgt_tensor == pad_id).to(
        device)
    src_padding_mask = (src == pad_id).to(device)

145 x = model.pos_encoder(model.embedding(
    tgt_tensor))
    for layer in model.decoder_layers:
        x = layer(x, memory, tgt_mask,
            tgt_padding_mask, src_padding_mask)

    logits = model.out_linear(x[:, -1, :])
150 next_token = torch.argmax(logits, dim=-1).item
    ()

    if next_token == eos_id:
        break
    tgt_tokens.append(next_token)

155 hyp_text = tokenizer.decode(tgt_tokens[1:])

errors.append({
    'src': src_text,
160 'ref': ref_text,
    'hyp': hyp_text
})

for i, error in enumerate(errors, 1):
165 print(f"\nПример {i}:")
    print(f"    Исходник: {error['src']}")
    print(f"    Референс: {error['ref']}")
    print(f"    Перевод: {error['hyp']}")

```

```
170         return errors
    analyze_errors(model, val_dataloader, num_examples=10)
```